

Report on Assignment 3

CUDA Programming

Students:

Behnoud Shafiezadeh Kenari
(5655740)

Shayan Alvansazyazdi
(5447411)

Sina Hatami
(5447389)

Feb 2024

Experiments

We have implemented our code with GPU in Google Colab (Tesla T4) and CPU of it.

After implementing the thread size `<<<10,10>>>` and `<<<30,30>>>` and combining them with different input sizes such as 1000*1000, 5000*5000, and 10000*10000 (dimensions of our input matrices), the following run times based on Second have been resulted. Consequently, we can observe, the following result:

Compiling step of code with thread size `<<<10,10>>>`:

```
✓ 2s [5] # with grid size <<<10,10>>>
      !nvcc -o home_work home_work.cu -lm
```

- With matrice 1000*1000:

```
✓ 3s [6] !./home_work 1000 1000

input matrix dimensions: 1000 x 1000
CPU Version, Elapsed time (s) = 2.14
GPU Version, Elapsed time (s) = 0.13
The Max Error of 0.00001 is within acceptable bounds.
```

- With matrices 5000*5000:

```
✓ 1m [7] !./home_work 5000 5000

input matrix dimensions: 5000 x 5000
CPU Version, Elapsed time (s) = 60.01
GPU Version, Elapsed time (s) = 0.37
The Max Error of 0.00002 is within acceptable bounds.
```

- , and finally 10000*10000:

```
✓ 4m [8] !./home_work 10000 10000

input matrix dimensions: 10000 x 10000
CPU Version, Elapsed time (s) = 246.63
GPU Version, Elapsed time (s) = 1.26
The Max Error of 0.00002 is within acceptable bounds.
```

As a result, by increasing the size of dimension, the CPU version has considerably more executable run time than GPU version, meaning that GPU kernels have more speed than CPU cores. However, by increasing the dimension, both GPU and CPU run times are increasing as they process more data in matrice than lower dimension ones.

And, in another scenario, we have experimented with the thread size <<<32,32>>>. So, firstly, we compile the Cuda code, with the below instruction:

```
✓ 1s [9] # with grid size <<<32,32>>>
      !nvcc -o Cuda_Homework Cuda_Homework.cu -lm
```

- With matrix 1000*1000:

```
✓ 3s !./Cuda_Homework 1000 1000

input matrix dimensions: 1000 x 1000
CPU Version, Elapsed time (s) = 2.17
GPU Version, Elapsed time (s) = 0.00
Problem! The Max Error of 6.16635 is NOT within acceptable bounds.
```

- With matrices 5000*5000:

```
✓ 1m [11] !./Cuda_Homework 5000 5000

input matrix dimensions: 5000 x 5000
CPU Version, Elapsed time (s) = 59.24
GPU Version, Elapsed time (s) = 0.00
Problem! The Max Error of 6.26814 is NOT within acceptable bounds.
```

- , and finally 10000*10000:

```
✓ 4m !./Cuda_Homework 10000 10000

input matrix dimensions: 10000 x 10000
CPU Version, Elapsed time (s) = 239.49
GPU Version, Elapsed time (s) = 0.00
Problem! The Max Error of 6.23431 is NOT within acceptable bounds.
```

As we expected the run time of CPU version should not be changed as before, but the GPU version of code by increasing the thread size from <<<10,10>>> to <<<32,32>>>, the run times are converged to 0 Second, meaning that this parallel version code has been implemented as fast as we have expected, that is incredible. In fact, it has most significantly higher speed than the before version with lower threads.

Description of the parallelized version of code with Cuda

We describe only the code parts relating to the parallelization part with the Cuda library.

// Kernel Function Definition (__global__ void step_kernel_mod(...)):

- The __global__ keyword indicates that this function is a CUDA kernel, and it can be called from the CPU but executed on the GPU.
- The function takes several parameters: ni, nj, fact, temp_in, and temp_out.

```
__global__ void step_kernel_mod(int ni, int nj, float fact, float* temp_in, float* temp_out)
{
```

// i and j are calculated based on the block and thread indices.

```
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
```

// Allocate shared memory

// A 2D shared memory array shared_temp is allocated with size [BLOCK_SIZE_X + 2][BLOCK_SIZE_Y + 2] to store a portion of the input data.

```
    __shared__ float shared_temp[BLOCK_SIZE_X + 2][BLOCK_SIZE_Y + 2];
```

// Global indices

// gi and gj are global indices obtained from the block and thread indices.

```
    int gi = i + blockIdx.x * blockDim.x;
    int gj = j + blockIdx.y * blockDim.y;
```

// Shared memory indices

```
    int si = threadIdx.x + 1;
    int sj = threadIdx.y + 1;
```

// Copy data to shared memory

// The input data (temp_in) is copied into the shared memory.

// Boundary data is loaded into the shared memory to handle edge cases. This involves fetching neighboring elements from global memory and storing them in the shared memory.

```
    shared_temp[si][sj] = temp_in[I2D(ni, gi, gj)];
```

// Load boundary data

// The input data (temp_in) is copied into the shared memory.

// Boundary data is loaded into the shared memory to handle edge cases. This involves fetching neighboring elements from global memory and storing them in the shared memory.

```
    if (threadIdx.x == 0 && i > 0) {
        shared_temp[si - 1][sj] = temp_in[I2D(ni, gi - 1, gj)];
    }
    if (threadIdx.x == blockDim.x - 1 && i < ni - 1) {
        shared_temp[si + 1][sj] = temp_in[I2D(ni, gi + 1, gj)];
    }
    if (threadIdx.y == 0 && j > 0) {
        shared_temp[si][sj - 1] = temp_in[I2D(ni, gi, gj - 1)];
    }
    if (threadIdx.y == blockDim.y - 1 && j < nj - 1) {
        shared_temp[si][sj + 1] = temp_in[I2D(ni, gi, gj + 1)];
    }
```

```

}

// Synchronize threads to ensure data is loaded into shared memory
// This ensures that all threads in the block have finished loading data into the shared memory before
// proceeding with further computations.

__syncthreads();

// Conditioning to avoid out-of-bounds error
// The code calculates second-order partial derivatives (d2tdx2 and d2tdy2) using the shared memory
// data.
// The final temperature update is performed using the finite difference method, incorporating the
// computed derivatives and a scaling factor (fact).

// An if-statement ensures that the computation is only performed for valid indices, avoiding out-of-
// bounds errors.

if ((gi > 0 && gi < ni - 1) && (gj > 0 && gj < nj - 1)) {
    int i00 = I2D(ni, gi, gj);

    // Evaluate derivatives using shared memory
    float d2tdx2 = shared_temp[si - 1][sj] - 2 * shared_temp[si][sj] + shared_temp[si + 1][sj];
    float d2tdy2 = shared_temp[si][sj - 1] - 2 * shared_temp[si][sj] + shared_temp[si][sj + 1];

    // Update temperatures
    temp_out[i00] = shared_temp[si][sj] + fact * (d2tdx2 + d2tdy2);
}
}

```

Additionally, in main() function we have some changes which they should be checked:

```

// cudaMalloc is a CUDA function used to allocate memory on the device (GPU).
// The first argument is the pointer to the pointer that will store the allocated memory address.
// &dev_temp1 and &dev_temp2 are the addresses of pointers pointing to the device memory for
// temperature data.
// size is the total size of the memory to allocate, calculated as the product of the dimensions ni and nj
// (size = ni * nj * sizeof(float)).

```

```

cudaMalloc((void**)&dev_temp1, size);
cudaMalloc((void**)&dev_temp2, size);

```

```

// Time Measurement in CUDA

```

```

// cudaEvent_t is a CUDA type representing an event that can be recorded and used for measuring
// time on the GPU.
// cudaEventCreate(&start) and cudaEventCreate(&end) are used to create CUDA events for
// recording the start and end times.
// float elapsedTime; declares a variable to store the elapsed time between the start and end events.

```

```

cudaEvent_t start, end;
cudaEventCreate(&start);
cudaEventCreate(&end);
float elapsedTime;

```

```
// Define the number of blocks and threads per blocks
// dim3 is a CUDA type representing a three-dimensional variable (x, y, and z dimensions).
// threads(32, 32) creates a dim3 variable named threads with x and y dimensions set to 32 each. This
// specifies the number of threads per block in a two-dimensional grid.
// blocks is another dim3 variable representing the number of blocks in the grid. The number of blocks
// is calculated based on the dimensions (ni and nj) of the problem and the number of threads per block.
// (ni + threads.x - 1) / threads.x calculates the number of blocks along the x-dimension, ensuring that
// it covers the entire input array. It performs the ceiling division, making sure that if ni is not exactly
// divisible by threads.x, an extra block is added to cover the remainder.
// (nj + threads.y - 1) / threads.y does the same for the y-dimension.
```

```
dim3 threads(32, 32);
dim3 blocks ((ni + threads.x-1)/threads.x, (nj + threads.y-1)/threads.y);
```

```
//Copy the from host to device.
// cudaMemcpy is a CUDA function for memory copy operations between the host and the device.
// dev_temp1 and dev_temp2 are pointers to device memory where temperature data is stored on the
// GPU.
// temp1_mod and temp2_mod are pointers to host memory where the initial temperature data is
// stored on the CPU.
// size represents the number of bytes to copy and is calculated as the product of the dimensions (ni
// and nj) and the size of each element (sizeof(float)).
// cudaMemcpyHostToDevice is an enumerated value that specifies the direction of the memory copy,
// indicating that the data is being transferred from the host to the device.
```

```
cudaMemcpy(dev_temp1, temp1_mod, size, cudaMemcpyHostToDevice);
cudaMemcpy(dev_temp2, temp2_mod, size, cudaMemcpyHostToDevice);
```

```
// The loop iterates over the specified number of time steps (nstep), which determines how many times
// the heat diffusion computation will be performed on the GPU.
// step_kernel_mod<<<blocks, threads>>>(ni, nj, tfac, dev_temp1, dev_temp2); launches the CUDA
// kernel step_kernel_mod with the specified grid and block dimensions (blocks and threads) and passes
// the
// necessary parameters (ni, nj, tfac, dev_temp1, dev_temp2). This is the point where the parallel
// computation on the GPU takes place.
// After each iteration, the temperature pointers on the GPU (dev_temp1 and dev_temp2) are swapped.
// This is done using temporary pointers (dev_tmp). This swapping ensures that the updated
// temperatures from the current iteration become the input for the next iteration.
```

```
for (istep=0; istep < nstep; istep++) {

    step_kernel_mod<<<blocks, threads>>>(ni, nj, tfac, dev_temp1, dev_temp2);
    // swap the temperature pointers
    dev_tmp = dev_temp1;
    dev_temp1 = dev_temp2;
    dev_temp2= dev_tmp;

}
```

```
//copy from device to host
cudaMemcpy(temp1_mod, dev_temp1, size, cudaMemcpyDeviceToHost);
```

```
// cudaEventRecord(end, 0);: Records the current time into the CUDA event end.  
// cudaEventSynchronize(end);: Ensures that the CPU waits for the GPU to finish recording the end  
event. This synchronization is necessary to get accurate timing measurements.  
// cudaEventElapsedTime(&elapsedTime, start, end);: Calculates the elapsed time between the start  
and end events and stores it in the variable elapsedTime. The time is measured in milliseconds.  
// printf("GPU Version, Elapsed time (s) = %.2lf\n", elapsedTime/1000);: Prints the elapsed time in  
seconds, converting it from milliseconds.  
// cudaEventDestroy(start); and cudaEventDestroy(end);: Destroy the CUDA events start and end to  
release associated resources.
```

```
cudaEventRecord(end, 0);  
cudaEventSynchronize(end);  
cudaEventElapsedTime(&elapsedTime, start, end);  
printf("GPU Version, Elapsed time (s) = %.2lf\n", elapsedTime/1000);  
cudaEventDestroy(start);  
cudaEventDestroy(end);
```

Noticeably, other parts of code which are not discussed here remained stable concerning the first version of it which was serialized.